listing 5.2 works. This is also the basis for the higher-level synchronization facilities in the C++ Standard Library, such as mutexes and condition variables. To see how this works, consider the simple spin-lock mutex from listing 5.1.

The `lock()` operation is a loop on `flag.test_and_set()` using `std::memory_order_acquire` ordering, and the `unlock()` is a call to `flag.clear()` with `std::memory_order_release` ordering. When the first thread calls `lock()`, the flag is initially clear, so the first call to `test_and_set()` will set the flag and return `false`, indicating that this thread now has the lock, and terminating the loop. The thread is then free to modify any data protected by the mutex. Any other thread that calls `lock()` at this time will find the flag already set and will be blocked in the `test_and_set()` loop.

When the thread with the lock has finished modifying the protected data, it calls `unlock()`, which calls `flag.clear()` with `std::memory_order_release` semantics. This then synchronizes-with (see section 5.3.1) a subsequent call to `flag.test_and_set()` from an invocation of `lock()` on another thread, because this call has `std::memory_order_acquire` semantics. Because the modification of the protected data is necessarily sequenced before the `unlock()` call, this modification happens-before the `unlock()` and thus happens-before the subsequent `lock()` call from the second thread (because of the synchronizes-with relationship between the `unlock()` and the `lock()`) and happens-before any accesses to that data from this second thread once it has acquired the lock.

Although other mutex implementations will have different internal operations, the basic principle is the same: `lock()` is an acquire operation on an internal memory location, and `unlock()` is a release operation on that same memory location.

5.4. Summary

In this chapter I've covered the low-level details of the C++11 memory model and the atomic operations that provide the basis for synchronization between threads. This includes the basic atomic types provided by specializations of the `std::atomic<>` class template as well as the generic atomic interface provided by the primary `std::atomic<>` template, the operations on these types, and the complex details of the various memory-ordering options.

We've also looked at fences and how they can be paired with operations on atomic types to enforce an ordering. Finally, we've come back to the beginning with a look at how the atomic operations can be used to enforce an ordering between nonatomic operations on separate threads.

In the next chapter we'll look at using the high-level synchronization facilities alongside atomic operations to design efficient containers for concurrent access, and we'll write algorithms that process data in parallel.